Práctica 2

Metaheurísticas

Juan Antonio Mellado Arenas
Nicolás Sánchez Álvarez
Antonio Jesús Merlo Morales
Sergio Palacios López
Sergio García Atanasio

Universidad de Córdoba

Índice

1 Introducción **2**

2 Enunciado de la práctica **2**

3 Representación y Función Objetivo **2**
3.1 Representación 2
3.2 Función objetivo 3

4 Población inicial **3**

5 Selección padres **4**

6 Cruces entre individuos **4**

7 Mutación de individuos **6**

8 Reemplazo **6**

9 Probabilidades de mutación y cruce **6**

10 Criterios de parada **7**

11 Consideraciones extra **8**
11.1 Control de dominios 8
11.2 Normalización y cálculo de distancias 8

12 Grid Search, Random Search y Algoritmo genético **9**
12.1 Grid Search 9
12.2 Random Search 10

13 Análisis resultados **10**
13.1 Grid Search 10
13.2 Random Search 11
13.3 Algoritmo Genético 11

14 Conclusión **12**

1 Introducción

En este documento, se muestran sendos proceso y resultado del desarrollo de la segunda práctica de la asignatura de Metaheurísticas. El objetivo de esta práctica consiste en la implementación de un algoritmo genético para la optimización de hiperparámetros de un modelo de aprendizaje automático, así como su comparación entre los algoritmos Random Search y Grid Search. Cualquier aspecto relativo al código fuente, o información adicional queda en el repositorio de la práctica.

2 Enunciado de la práctica

La práctica plantea que, dado un conjunto real de datos (será el dataset Wine Quality), se ha de maximizar la precisión de clasificación del modelo de aprendizaje automático dado. Wine Quality considera datos como pH del vino, su acidez, densidad, entre otros. Sobre esto, se transforma la clasificación del problema tal que sea de carácter binario, resultando en que un vino puede ser bueno o malo según si su calidad supera o no supera 6.

Sobre el modelo Random Forest no se debe tomar consideraciones en cuanto a su funcionamiento por dentro, ya que, el objeto de la práctica es optimizar los hiperparámetros, y eso incluye una abstracción del modelo tal que se perciba como una herramienta capaz de evaluar esos parámetros y devolviendo una calidad.

Así pues, para lo que será el algoritmo genético, tenemos una codificación tal que un individuo es un vector con sus genes (parámetros) en orden. Algunos genes tienen un dominio binario, mientras que otros operan en un rango definido para los números reales.

Utilizando un paradigma orientado a objetos, se ha implementado la clase Individual, que tiene los genes como atributos, además de algunos campos ajenos a los genes que serán útiles a la hora de implementar el algoritmo, como la posición que ocupa cada individuo en la población. Haciendo uso del modelo, se obtiene un atributo score o puntuación, que es la media de los 5 folds como se indica en la práctica (de nuevo, esto no es relevante en el desarrollo del algoritmo). Una población es un conjunto de objetos de la clase individuo.

3 Representación y Función Objetivo

3.1 Representación

Lo primero que haremos será definir la representación que utilizaremos para aplicar nuestro algoritmo genético y abordar nuestro problema principal. La representación más sencilla que podemos utilizar es una de tipo lineal, en este caso, combinada con valores enteros y binarios. Para aplicarlo en Python, crearemos una clase con los siguientes campos.

Estos son los atributos que aparecen en nuestra clase individuo, a efectos prácticos trabajaremos con ella como si fuese un vector de parámetros. Los dos últimos parámetros, position y score, no se utilizarán a la hora de realizar operaciones de selección, cruce o mutación.

El parámetro position indica la posición que tiene cada individuo en la población, cosa que será bastante útil cuando necesitemos hacer reemplazamiento. El parámetro score indica el valor que devuelve la función de fitness, y se calcula únicamente cuando se crea el individuo para ahorrar cálculos innecesarios (también se recalcula cuando se hace una operación de mutación, por ejemplo).

```
class Individual:
  def __init__(
      self,n_estimators,max_depth,min_samples_split,min_samples_leaf,max_features,
  bootstrap,
      criterion,class_weight,max_leaf_nodes,min_impurity_decrease,random_state,
  position,
  ):
      self.n_estimators = n_estimators
      self.max_depth = max_depth
      self.min_samples_split = min_samples_split
      self.min_samples_leaf = min_samples_leaf
      self.max_features = max_features
      self.bootstrap = bootstrap
      self.criterion = criterion
      self.class_weight = class_weight
      self.max_leaf_nodes = max_leaf_nodes
      self.min_impurity_decrease = min_impurity_decrease
      self.random_state = random_state
      self.position = position
      self.score = model.evaluate_solution(self)
```

Listing 1: Constructor de la clase Individuo

3.2 Función objetivo

La función objetivo o fitness será el valor de accuracy que devuelve el modelo una vez ha sido entrenado con los hiperparámetros que obtenemos a partir del individuo. Como se ha explicado, en este caso actúa como "una caja negra", de manera que no requiere de implementación, y el objetivo será simplemente maximizar la puntuación.

4 Población inicial

El comienzo del algoritmo genético es, la creación de una población inicial sobre la que iterar el resto de las etapas de este. Para el algoritmo se ha implementado una generación de población inicial secuencial. Esta consiste en crear individuos aleatorios, con el matiz de que los individuos distan entre sí una cantidad considerable de diferencias en sus genes. Al final resultará una población de la que partir con individuos que tienen genes aleatorios, pero que, de manera general son todos diferentes entre sí como mínimo una distancia definida.

5 Selección padres

Una vez que se tiene la población inicial, el próximo paso es escoger los padres de la población. Para esto el grupo ha decidido hacer uso de la selección por torneo. Esta técnica, es una de las más costosas y complejas, no obstante, resulta en mejores soluciones de forma general. Además, es un balance entre aleatoriedad y una selección de los mejores individuos.

La cuestión principal en selección por torneo es, el tamaño del torneo. Si se toma un tamaño muy grande surge alta presión selectiva, y al final únicamente pasan los individuos mejores, de forma repetida. Esto puede terminar en muy poca diversificación y en incesto.

Por eso nuestro tamaño de torneo es 3. Se seleccionan tres individuos aleatorios de la población, y se evalúan según su score, el individuo que más score tenga será el seleccionado para ser padre. La selección de individuos para enfrentarlos en el torneo es con reposición (pueden aparecer individuos repetidos)

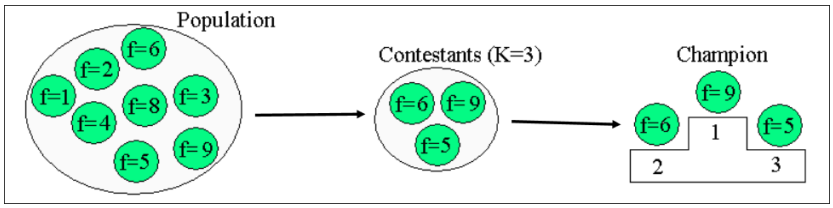

Figure 1: Selección por torneo, con K=3

Se ha implementado que además de los padres elegidos por torneo, los dos mejores individuos de cada generación sobrevivan y pasen a la próxima generación. Este fenómeno contribuye a la mejora de la media de la puntuación del algoritmo.

6 Cruces entre individuos

Una vez escogidos los padres de la población, la etapa que sigue consiste en cruzar dichos individuos para obtener nuevos individuos hijos. Para efectuar este proceso, se ha desarrollado un operador de cruce híbrido, que combina dos estrategias distintas: cruce uniforme y cruce en n puntos.
El cruce uniforme consiste en generar un nuevo individuo seleccionando, para cada gen, el valor de uno de los dos progenitores de forma independiente y aleatoria. De este modo, cada parámetro del hijo proviene de uno de los padres con una cierta probabilidad, favoreciendo la exploración del espacio de soluciones.

Por otro lado, el cruce en n puntos es una técnica en la que se seleccionan varios puntos de corte a lo largo del cromosoma. A partir de estos puntos, se alternan bloques de genes entre ambos progenitores para construir el descendiente. En este caso se han utilizado 4 puntos de corte. Este método permite preservar combinaciones parciales de parámetros que pueden ser beneficiosas, favoreciendo la explotación.

Ambos operadores generan dos descendientes a partir de cada pareja de padres.
El algoritmo activa el cruce si la probabilidad de cruce lo permite (este mecanismo se detalla más

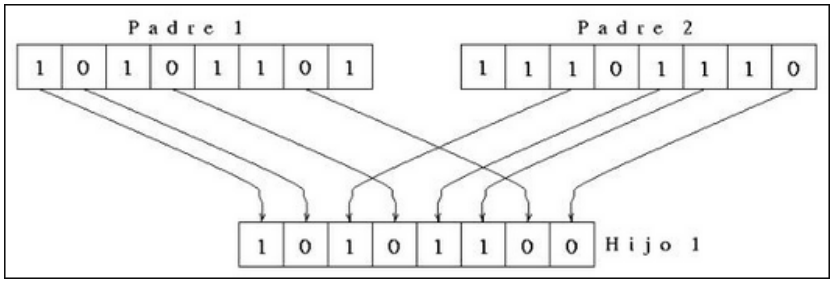

Figure 2: Cruce uniforme

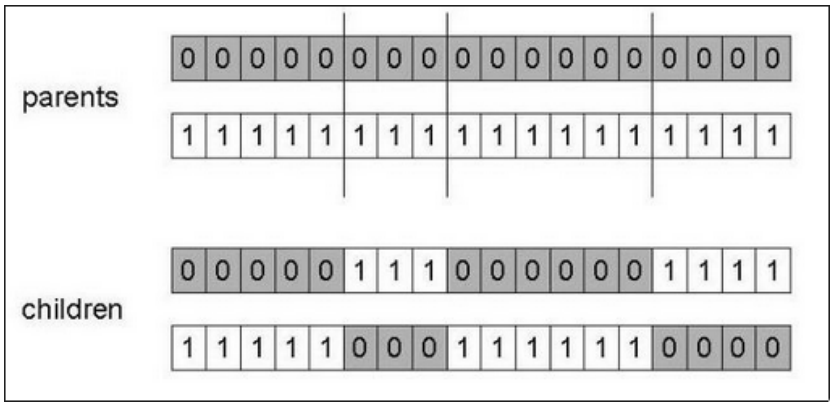

Figure 3: Cruce de n-puntos

adelante en el documento), y en caso afirmativo, se selecciona de manera aleatoria cuál de los dos operadores aplicar, asignando aproximadamente la misma probabilidad a cada uno. De este modo, la mitad de los cruces se realizan mediante cruce uniforme y la otra mitad mediante cruce en n puntos.

Las fortalezas del cruce híbrido frente a únicamente utilizar uno de los dos mecanismos explicados, son especialmente respecto al equilibrio de exploración y explotación, además, el algoritmo ahora tiene más variabilidad, lo que reduce la convergencia prematura. Sus debilidades son principalmente el costo ya que hay más evaluaciones, y al hacer dos hijos por cruce se deben entrenar todos estos, causando que el algoritmo tarde más. Además, la implementación ha resultado más compleja que utilizar un único operador de cruce.

Para implementar este método, se han normalizado los genes entre 0 y 1. Posteriormente se hace uso de la distancia euclidiana, lo que permite definir como de alejados se encuentran dos individuos según sus genes. Para más información, revisar la sección 11.2.

El costo de implementar esta estrategia y ejecutarla a nivel computacional es medio, y su punto más débil es que las soluciones iniciales no serán de gran calidad (al estar creados los individuos de manera aleatoria). Con todo esto sigue siendo un método idóneo por su balance y, teniendo en cuenta que la mejora de soluciones es algo que se obtendrá posteriormente en el algoritmo, y que, en este punto interesa diversificar lo máximo posible.

7 Mutación de individuos

La mutación consiste en modificar ligeramente ciertos individuos. En este caso, se aplica una mutación uniforme sobre todos los genes del individuo. El operador recorre cada gen con una determinada probabilidad de modificación y, si esta se cumple, el gen se reemplaza por un valor aleatorio dentro del dominio permitido para dicho parámetro.

Para garantizar que, al activarse la mutación, el individuo experimente al menos un cambio (ya que podría darse el caso de que ningún gen fuese modificado), el operador fuerza la modificación de al menos uno de los genes. Tras la mutación, se actualiza el *score* del individuo.

Esta estrategia resulta efectiva para introducir nuevos valores que no estaban presentes previamente en la población, reduciendo el riesgo de convergencia prematura y favoreciendo la exploración de nuevas regiones del espacio de búsqueda. De hecho, en situaciones de estancamiento, la mutación puede permitir alcanzar soluciones mejores que no se obtendrían únicamente mediante el cruce.

A diferencia de este último, se ha optado por un operador de mutación único y más sencillo. No obstante, su principal desventaja es que puede deteriorar soluciones con una puntuación elevada.

8 Reemplazo

Tras la mutación, el algoritmo genético realiza la técnica de reemplazo del peor individuo (o individuos). Este ordena primero la población según el valor obtenido al evaluar la solución, y dependiendo del número de individuos que vayan a ser reemplazados en la población, se seleccionan aquellos individuos para su reemplazo.

En este caso, se ha tenido en cuenta si la población necesita diversificar más o menos. Para cuando es necesario intensificar, se reemplazarán a los n peores de la población con los mejores hijos obtenidos, pero en el caso de que sea necesario diversificar más, se reemplazará a los n peores de la población con un individuo aleatorio, para que existan individuos más diferentes.

Se pensó en un primer momento en utilizar crowding determinístico ya que es muy útil a la hora de mantener la diversidad en la población, pero no arrojó resultados idóneos, ya que en nuestro caso no llegaba a converger lo rápido que queríamos, y la varianza no variaba mucho, por lo siempre aumentaba o disminuía una de las probabilidades de cruce o de mutación (siguiente apartado).

9 Probabilidades de mutación y cruce

Se ha tenido en cuenta la variabilidad de la probabilidad de mutación y la probabilidad de cruce. Para ello, el algoritmo calcula por cada generación la media de score de toda la población, y la varianza de los parámetros de los individuos que se encuentran en la población, normalizando los individuos como se ha descrito anteriormente.

No hemos utilizado la varianza del score de la población, ya que realmente un individuo con unos parámetros diferentes al de otro individuo puede tener el mismo score. De esta forma, comprobamos realmente la variabilidad que existe en la población.

Primero hemos comenzado inicializando las probabilidades de mutación y de cruce a 0.5. Después, en cada generación dependiendo de un valor de varianza predefinido (tras varias ejecuciones, se ha determinado que en torno a 0.12 es un valor idóneo).

- Si la varianza es menor que este valor predefinido, significa que los individuos de la población son muy parecidos entre ellos, por lo que necesitamos diversificar, y con ello aumentar la probabilidad de mutación y disminuir la de cruce.
- Si la varianza es mayor que el valor de threshold, significa que los individuos son lo suficientemente diferentes como para poder intensificar, así que aumentaremos el valor de la probabilidad de cruce y disminuiremos la de mutación

Dicho esto, lo usual es que al principio vaya aumentando mucho la probabilidad de cruce, hasta que llegue un punto que la varianza es tan baja, que debería empezar a aumentar la probabilidad de mutación y disminuir la de cruce, hasta llegar a un tope. Un ejemplo de una ejecución se puede observar en la siguiente gráfica:

iteraciones	media˙score	varianza˙params	prob˙mut	prob˙cross
1	0.7028140510710553	0.13105116796307203	0.4	0.6
2	0.7289481289184954	0.13884470396281984	0.30000000000000004	0.7
3	0.7371556622257053	0.128308289906175	0.20000000000000004	0.7999999999999999
4	0.7372606452455589	0.1160213569315124	0.30000000000000004	0.7
5	0.7292331178160921	0.12611999786122535	0.20000000000000004	0.7999999999999999
6	0.73361391392372	0.10913515483665139	0.30000000000000004	0.7
7	0.7295509241118078	0.14706005928755064	0.20000000000000004	0.7999999999999999
8	0.7328609913793103	0.10205941479743767	0.30000000000000004	0.7
9	0.73048891392372	0.1020139208578844	0.4	0.6
10	0.7328066222570532	0.08846796140221659	0.5	0.5
11	0.7249912650208987	0.11329043457262836	0.6	0.4
12	0.7241814426593521	0.12084454595059614	0.5	0.5
13	0.7235560344827586	0.12232828585044904	0.4	0.6
14	0.7264472309299895	0.12106574331028173	0.30000000000000004	0.7
15	0.7350957582288403	0.09981025852975751	0.4	0.6
16	0.7284006008359457	0.10386981148872482	0.5	0.5
17	0.7285570957419018	0.10892883376974954	0.6	0.4
18	0.7223534646029258	0.12170766111266987	0.5	0.5
19	0.727879032784744	0.10586170175849068	0.6	0.4
20	0.7236584051724139	0.11310044956276966	0.7	0.30000000000000004

Podemos observar, como en las últimas iteraciones va aumentando la probabilidad de mutación y disminuyendo la de cruce, como habíamos predicho.

10 Criterios de parada

Se han considerado los siguientes criterios para detener la evolución de la población:

- **Número máximo de generaciones**: Para evitar que se ejecute en exceso, en el algoritmo genético en cuestión se considera un máximo de 24 generaciones. Se ha observado que este

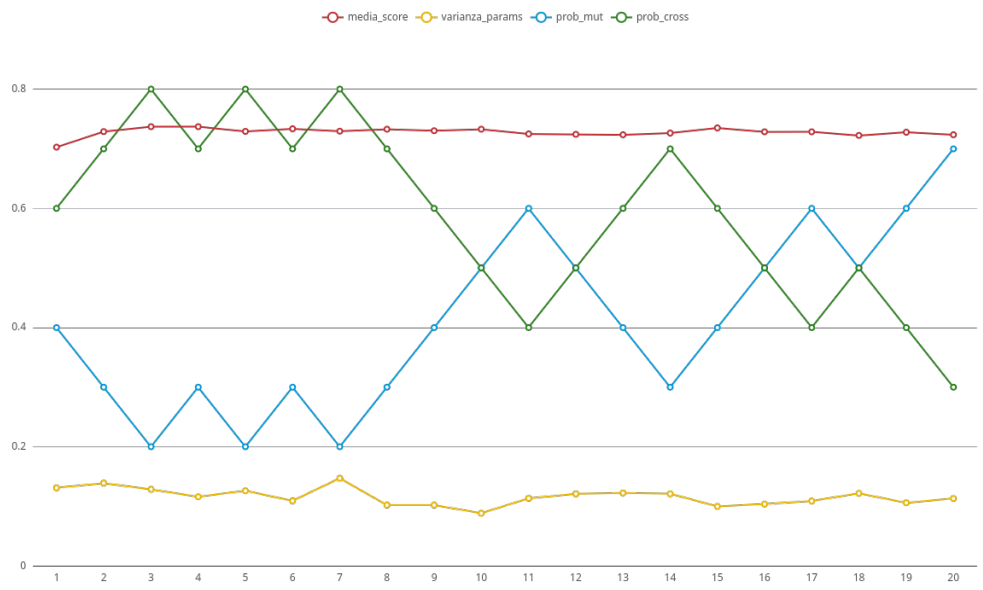

Figure 4: Probabilidades de mutación y cruce tras 20 iteraciones

número de iteraciones es suficiente como para inferir conclusiones sin suponer demasiado tiempo de ejecución.

- **Inversión de probabilidades**: Además del límite físico de 24 iteraciones, se ha implementado un criterio de parada basado en la convergencia evolutiva que detiene el algoritmo cuando se da la inversión de probabilidades de operadores $P_{mut} > P_{cross}$.

11 Consideraciones extra

11.1 Control de dominios

Se han de tener en cuenta los valores de los parámetros de cada individuo y que se mantengan dentro de su dominio correspondiente en los cruces y las mutaciones.

Debido a que los cruces implementados no modifican los valores de los parámetros, no hay que tomar ninguna consideración de seguridad en este aspecto.

La única mutación implementada es la uniforme, se garantiza que los valores se mantengan dentro de los dominios definiendo el rango de la función de generación aleatoria.

11.2 Normalización y cálculo de distancias

Para que el algoritmo gestione la diversidad de manera cuantitativa, se ha implementado un sistema de normalización en el rango $[0, 1]$ para cada gen del individuo.

Debido a que los hiperparámetros del modelo operan en escalas distintas, una distancia calculada sobre los valores reales daría un peso excesivo a las variables con más rango. La función `NormaliseIndividual` transforma cada gen para normalizarlo:

- **Variables numéricas de rango definido**: Se escalan dividiendo el valor por el límite máximo permitido.
- **Variables categóricas y booleanas**: Se mapean directamente a 0.0 o a 1.0.
- **Manejo de valores especiales**: Se mapean a valores intermedios (`"sqrt" -> 0.5, "log2" -> 0.3`) para permitir su comparación matemática.

12 Grid Search, Random Search y Algoritmo genético

En adición al algoritmo genético, se han implementado una búsqueda aleatoria y grid search para comparar ambos métodos. De esta forma, podremos observar cómo se comporta nuestro algoritmo frente a opciones más clásicas, y si de alguna manera es mejor o peor.

12.1 Grid Search

Grid search es una técnica de búsqueda exhaustiva de los parámetros objeto del problema. Se definen unas divisiones (grid) con los posibles valores de los parámetros, y sobre estos valores se generan combinaciones.

En este caso, se ha definido una lista de la división de esos parámetros con los posibles valores:

- **n_estimators**: 10, 155, 300
- **max_depth**: 2, 15, 30
- **min_samples_split**: 2, 10, 20
- **min_samples_leaf**: 1, 10, 20
- **max_features**: 0.1, 0.5, 1
- **bootstrap**: True, False
- **criterion**: *gini*, *entropy*
- **class_weight**: *balanced*, *None*
- **max_leaf_nodes**: 10, 100, 200
- **min_impurity_decrease**: 0, 0.05, 0.1

Sobre estas opciones se crean los individuos, para finalmente ordenarlos y obtener la mejor o mejores configuraciones posibles. La principal bondad es que dentro del espacio (grid) que hemos definido, se afinan soluciones muy buenas (se encuentra la mejor combinación posible en la rejilla), aunque, la decisión surge al elegir qué se considera como división ya que grid search es computacionalmente muy costoso, y podría llegar a ser inviable.

Esto es fundamental para comprender los resultados así como la comparativa con los otros dos métodos relativos a esta práctica.

12.2 Random Search

La búsqueda aleatoria consiste en generar individuos (soluciones) de manera estocástica, tal que por cada gen (parámetro) se inicializa en un valor aleatorio. Este proceso se itera repetidas veces.

Este método es de los más rápidos que hay, pero como observaremos a continuación, la gran desventaja que tiene es la inconsistencia de los datos obtenidos por cada ejecución del algoritmo. Como resultado, la varianza debería ser bastante más elevada.

13 Análisis resultados

Con todos los métodos ya implementados, la fase que sigue consiste en evaluar y comparar los resultados que han aportado tras su ejecución. Para ello, se han tenido en cuenta diferentes métricas, como las puntuaciones máximas, mínimas y su promedio. Además, se estudian la varianza y desviación de los resultados. Finalmente, en cuanto a la eficiencia y coste de los métodos tratados en la práctica, hemos hallado el tiempo de la ejecución en segundos, los MB de memoria y el porcentaje medio del procesador utilizado durante la ejecución.

13.1 Grid Search

Para el grid search, probaremos todas las posibilidades dejando fijos únicamente 3 valores diferentes por cada campo del individuo. En concreto, estos son los parámetros que estableceremos para luego hacer todas las combinaciones posibles:

```
param_grid = {
      "n_estimators": [10, 155, 300],
      "max_depth": [2, 15, 30],
      "min_samples_split": [2, 10, 20],
      "min_samples_leaf": [1, 10, 20],
      "max_features": [0.1, 0.5, 1],
      "bootstrap": [True, False],
      "criterion": ["entropy", "gini"],
      "class_weight": ["balanced", None],
      "max_leaf_nodes": [10, 100, 200],
      "min_impurity_decrease": [0, 0.05, 0.1],
}
```

Listing 2: Grid de parámetros

El total de combinaciones posibles para la rejilla establecida, son 17496 individuos. Para hacer una comparación justa, estudiaremos los siguientes algoritmos basándonos en el mismo número de individuos. Los datos obtenidos tras el estudio son los siguientes:

Algoritmo	Varianza	Desviacion˙Tipica	Tiempo˙Segundos	Num˙Combinaciones	Media˙Accuracy	Max˙Accuracy	Min˙Accuracy	CPU˙Media˙%	Memoria˙MB
Grid Search	0.004219	0.064955	12317.82	17496	0.6581	0.7405	0.5347	101.35	178.62

Los resultados con respecto al uso de CPU Media, es un valor mayor a 100% porque suma el porcentaje de todos los hilos del procesador (es decir, con un procesador de 16 cores, el porcentaje máximo sería el 1600%). Vemos que Los resultados tienen sentido, pues al ser un algoritmo secuencial, lo más probable es que un hilo esté al 100% de uso y el resto apenas tenga uso.

El resto de resultados los podremos comparar con los otros algoritmos en la conclusión final, aunque podemos ir observando que la desviación típica es la mayor de todas, y la media de accuracy es de las más bajas con respecto a otros algoritmos. Al ser el grid search todas combinaciones diferentes entre sí, es normal que tenga una varianza y desviación típica mayor al resto, ya que es imposible que algún valor se repita.

13.2 Random Search

Como hemos recalcado en el apartado anterior, con intención de hacer una comparación equitativa, la búsqueda aleatoria se ha ejecutado de forma análoga generando el mismo número de soluciones, los datos obtenidos han sido:

Algoritmo	Varianza	Desviacion˙Tipica	Tiempo˙Segundos	Num˙Combinaciones	Media˙Accuracy	Max˙Accuracy	Min˙Accuracy	CPU˙Media˙%	Memoria˙MB
Random Search	0.001822	0.042687	14744.9	17496	0.6797	0.7448	0.479	100.81	165.8

Primeramente podemos observar que hay una mejora en cuanto a los resultados de accuracy obtenidos, y un poco menos de varianza con respecto al anterior. La explicación es que, al poder obtener el mismo valor repetidas veces, la varianza tiende a ser menor debido a la posible repetición de configuraciones. Y el score obtenido medio y máximo, es mejor con respecto al grid, ya que grid search prueba todas las combinaciones posibles dentro de una rejilla, evaluando muchas combinaciones que obtienen un accuracy muy malo.

13.3 Algoritmo Genético

Finalmente, el algoritmo genético presenta una distinción a la hora de ejecutarlo. Se han de modificar sendos tamaño de la población y número de generaciones para equiparar la exhaustividad de los otros dos métodos. Así pues, hemos ejecutado el algoritmo genético implementado durante 349 generaciones. El tamaño de la población es cincuenta individuos. El resto de parámetros y decisiones de implementación/ejecución son los que se han explicado a lo largo de este documento. Los resultados han sido:

Algoritmo	Varianza	Desviacion˙Tipica	Tiempo˙Segundos	Media˙Accuracy	Max˙Accuracy	Min˙Accuracy	CPU˙Media˙%	Memoria˙MB
Genetico - Gen 1	0.0011	0.033166	70.59	0.6907	0.7361	0.5535	100.88	150.95
Genetico - Gen 2	0.000943	0.030714	113.31	0.7024	0.7367	0.6242	100.78	151.08
Genetico - Gen 3	0.000819	0.028613	174.76	0.7182	0.7367	0.6242	100.72	151.47
Genetico - Gen 4	0.000461	0.021477	246.6	0.7254	0.7417	0.6242	100.78	152.11
Genetico - Gen 5	0.000164	0.012809	314.95	0.7271	0.738	0.6754	100.73	152.16
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
Genetico - Gen 348	0.000174	0.013176	26484.1	0.7371	0.7449	0.676	100.79	173.07
Genetico - Gen 349	9.2e-05	0.009596	26560.8	0.7381	0.7436	0.6835	100.79	173.13

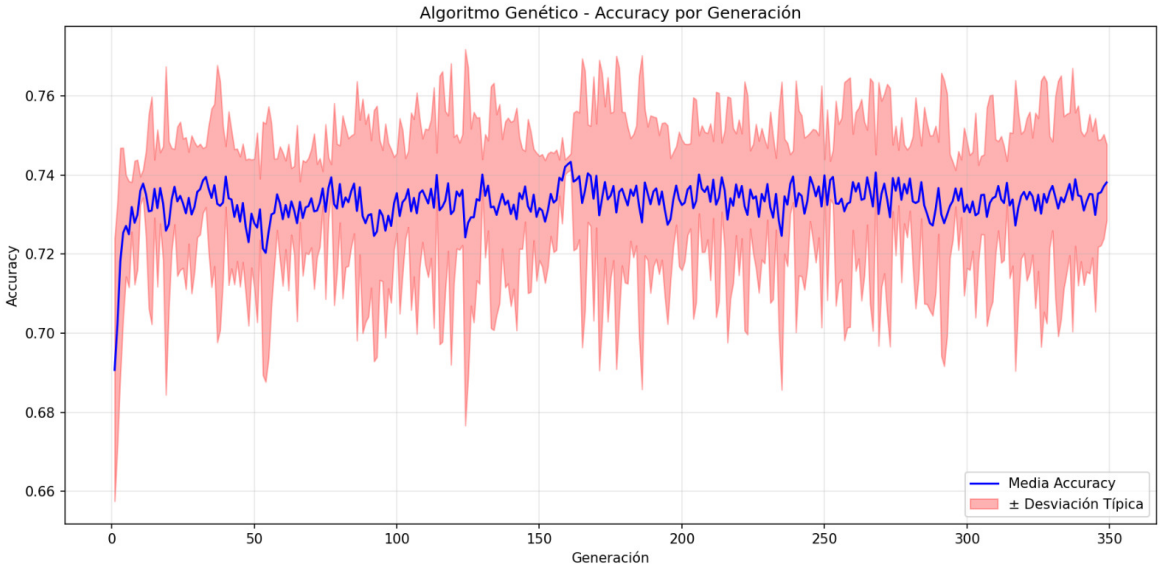

Figure 5: Valores obtenidos de Media y Varianza a lo largo de las generaciones

Nota: El csv completo con todos los datos está disponible en el repositorio de la práctica al igual que el resto de documentos y código fuente.

A continuación, se muestran unas gráficas con los resultados obtenidos:

Y el valor máximo de accuracy encontrado, en la generación 193:

Algoritmo	Varianza	Desviacion˙Tipica	Tiempo˙Segundos	Num˙Combinaciones	Media˙Accuracy	Max˙Accuracy	Min˙Accuracy	CPU˙Media˙%	Memoria˙MB
Genetico - Gen 193	0.000299	0.017299	14298.77	10243	0.7358	0.7517	0.6735	100.75	165.53

Podemos comprobar, que con respecto a los anteriores, es el algoritmo que más tiempo tarda (aproximadamente el doble de cada uno de los dos), lo cual es muy razonable al tener que hacer muchas más operaciones (cruce, mutación, selección, reemplazo, ordenaciones...). Aún así, sigue siendo el que mejor resultados nos ha dado, y con un uso de CPU y Memoria similares a los anteriores.

También es el que obtiene un valor de varianza más bajo, ya que al converger tan rápido la población, la gran mayoría de individuos de la población serán lo más parecidos entre sí, exceptuando aquellos que vayan mutando o se reemplacen los peores con aleatorios.

14 Conclusión

Tras el análisis comparativo de los modelos implementados, llegamos a la conclusión de que el Algoritmo Genético ha mostrado el mejor comportamiento para este problema por las siguientes razones fundamentales:

1. **CPU y Memoria:** El algoritmo genético es bastante similar a los anteriores en cuanto a uso de CPU, haciendo que no suponga una diferencia significativa en nuestro caso. En cuanto a Memoria prácticamente igual, aunque hay que tener en cuenta que Random Search va a utilizar menos memoria que Grid Search y el Algoritmo Genético (en nuestro caso, unos 10MB de diferencia).

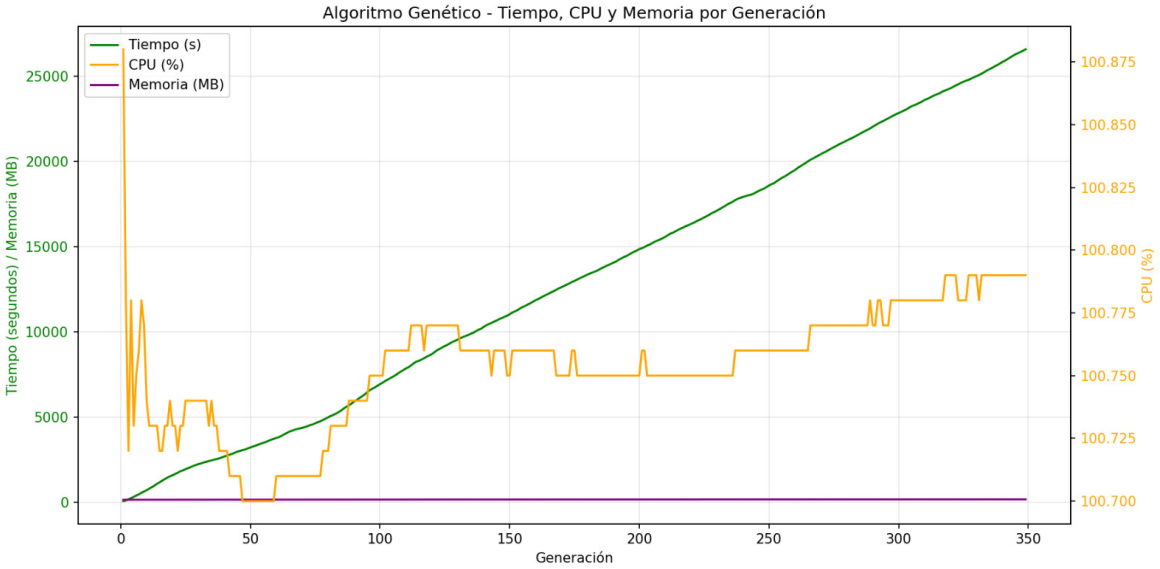

Figure 6: Valores obtenidos de Media de CPU y Memoria a lo largo de las generaciones

2. **Accuracy y Tiempo:** Es cierto que el Algoritmo Genético ha sido el que más se ha ejecutado con diferencia, pero esto es debido a que hemos querido hacer una comparación justa para evaluar el mismo número de individuos que con Grid Search y Random Search. Si observamos la primera gráfica, observamos que el algoritmo converge en apenas 4 o 5 generaciones, para así después explorar más, por lo que no nos sería tan efectivo ejecutar el algoritmo durante tanto tiempo.

   No solo eso, sino que además ha sido el que ha encontrado el mejor valor de Accuracy con respecto a los otros dos, alrededor de la mitad del tiempo, acercándose más al óptimo global con respecto a los otros dos.

3. **Punto intermedio:** Si analizamos detenidamente Grid Search y Random Search, nos damos cuenta de que son algoritmos opuestos, pues si probásemos absolutamente todas las combinaciones posibles con Grid Search, estaríamos realizando una búsqueda exhaustiva sobre todo el espacio de exploración. En cambio, Random Search al ser una búsqueda completamente aleatoria, estaría utilizando diversificación total.

   Dicho esto, con nuestro Algoritmo Genético podemos determinar un punto intermedio para así obtener unos resultados lo más cercanos al óptimo sin tener que explorar todo nuestro espacio de búsqueda y sin dejar el resultado completamente dependiente del azar.

4. **Valor aceptable de accuracy:** Aunque es cierto que nuestro algoritmo genético a la larga ha conseguido obtener el valor más cercano al óptimo, realmente no nos generaría una mejora tan significativa (según los datos obtenidos, más o menos un 0.01 de diferencia con respecto al valor usual: 0.74). Es por ello que podríamos detener el algoritmo a las 10 o 20 generaciones, y esperar tanto tiempo para ello. De hecho, si nos importase todavía menos esa diferencia de score, nos podría interesar más un Random Search, capaz de encontrar un valor relativamente decente sin

tener que explorar mucho, pero definitivamente no utilizaríamos un Grid Search si ese fuera el caso.